

BEKSULTANOV ASANALI - 20232014 REPORT FOR CODE

I decided to create report to explain some of my code that I thought could not be explain in comment line.

```
training_config = {
    "batch_size": 8,
    "epochs": 3,
    "lr": 1e-3,
    "warmup_steps": 50,
    "device": "cuda" if torch.cuda.is_available() else "cpu",

    "gradient_accumulate_steps": 1,
}
```

I tried to run the model with different batch sizes, learning rates and epochs and these hyperparameters gave me the best result. 1e-4 learning rate was too small in the beginning stage and the model simply did not reached it's local minimum with such a small learning rate, therefore I decided to change it to 1e-3.

```
attn_scores = torch.matmul(query, key.transpose(1, 2)) / self.scale

if encoder_attention_mask is not None:
    mask = encoder_attention_mask.unsqueeze(1)
    attn_scores = attn_scores.masked_fill(mask == 0, -1e9)

a = torch.softmax(attn_scores, dim=-1)
a = self.dropout(a)
context = torch.matmul(a, value)

output_context = self.output_proj(context)
return output_context
```

Context calculation as shown in the paper.

For Gather local context function:

After we calculate context word around which we should build window by this code:

```
p_t = self.location_proj_up(decoder_hidden_query)
p_t = self.location_proj_down(torch.tanh(p_t))
p_t = torch.sigmoid(p_t)
src_len = src_len.unsqueeze(2)
p_t = (src_len - 1) * p_t
p_t = p_t.squeeze(1) # (batch_size, 1)
```

```
src_len = src_len.squeeze(1) # (batch_size,1)

window_range = torch.arange(-self.window_size, self.window_size+1, device=device,
dtype=model_dtype) # (window_size * 2 + 1,)

window_positions = (p_t + window_range.unsqueeze(0)).long()
```

here we create window range [-win_size, . . . , 0, . . . +win_size], so and then we add broadcasting the position of our word so eventually having the word surrounded by its context words.

We also should consider the face that if win size and go out of bounds(below zero or out of our maximum sentence size) therefore we adding

```
window_positions = torch.clamp(window_positions,
                                min=torch.tensor(0, device=device),
                                max=src_len - 1).long()
```

```
additional_mask = torch.ones((batch_size, self.window_size * 2 + 1),
device=device, dtype=model_dtype)

zero_tensor = torch.tensor(0.0, device=additional_mask.device,
dtype=model_dtype)

additional_mask = torch.where(window_positions < 0, zero_tensor,
additional_mask)

additional_mask = torch.where(window_positions >= src_len.long(),
zero_tensor, additional_mask)
```

We should also consider for local mask that if the words is out of bounds (below zero or out of our maximum sentence size) we should also make them zero in local mask, because if additional mask is not applied, and word window is out of bounds, becuae of the clamp we will have this result - [0,0,0,0,1,2,3,4...], and so on and it gives the much more context for the word on zeros position instead of just not considering that tokens. Therefore we should consider that words in our local mask, and I implemented this by using torch.where that initially makes those tokens(below zero, and above our src_len) zero in the mask, even before clamping them.

```
local_attn_mask = local_attn_mask.float() * additional_mask
```

We then multiply our mask to the additional mask to get the full mask.

In generate function

```
logits, (encoder_outputs, current_h_dec, current_c_dec,
prev_attn_context) = self.forward(
    encoder_input_ids, encoder_attention_mask,
    decoder_input_ids[:, -1:], labels=None, cache=cache
)

cache = (encoder_outputs, current_h_dec, current_c_dec,
prev_attn_context)
```

We use this and only should consider the last token in decoder_input_ids, because of the cache, and the fact that we should not append the sequence fully.

Then we get next token from last position's logits, replace tokens for finished sequences with eos, append to sequence, and update unfinished flags

```
next_token = logits[:, -1, :].argmax(dim=-1) # (batch_size,)

next_token = next_token * unfinish_flag + eos * (1 - unfinish_flag)
next_token = next_token.unsqueeze(1) # (batch_size, 1)

decoder_input_ids = torch.cat([decoder_input_ids, next_token], dim=1)

unfinish_flag = unfinish_flag * (next_token.squeeze(1) != eos).long()
```